GUIDE 03

Backend Environment Setup in IntelliJ IDEA with Spring Boot MVC

UniMart Monolithic Full-Stack Lab Series

B.Sc. (Hons) IT - Level 2 | Hands-on Laboratory Practical

Department of Industrial Management
Faculty of Science, University of Kelaniya

Learning outcomes

- Create and run a Spring Boot monolithic REST API project in IntelliJ IDEA.
- Apply MVC and package-by-feature organization.
- Configure standard dependencies for JPA/Hibernate, validation, security, migrations and health checks.
- Establish controller, service and repository boundaries before CRUD implementation.

Prerequisites

- JDK 21 installed.
- IntelliJ IDEA Community or Ultimate.
- Maven and basic Java/OOP knowledge.

Suggested student time: 2-2.5 hours

1. Technology baseline

Item	Class baseline	Reason
Java	21 LTS	Stable long-term release and supported by current Spring Boot.
Spring Boot	Generate current stable 4.1.x with Spring Initializr	Starter versions are dependency-managed; avoid manual version mixing.
Build	Maven Wrapper	Same Maven execution on every student computer and CI runner.
Database access	Spring Data JPA + Hibernate	Repository abstraction and ORM requested for the lab.
Schema changes	Flyway	Repeatable, version-controlled database migrations.
Security	Spring Security + OAuth2 Resource Server JWT	Standard bearer-token validation without a custom filter parser.

2. Create the project using Spring Initializr

1. In IntelliJ choose New Project > Spring Boot, or use the official Spring Initializr.
2. Set Group: lk.ac.kln; Artifact: unimart-backend; Name: UniMart Backend; Package: lk.ac.kln.unimart.
3. Choose Maven, Java, packaging JAR and Java 21.
4. Add dependencies: Spring Web, Spring Data JPA, Validation, Spring Security, OAuth2 Resource Server, MySQL Driver, Flyway Migration, Actuator and Spring Boot Test.
5. Generate and open the project. Allow IntelliJ to import Maven dependencies.
6. Run ./mvnw -v and confirm Java 21 is used.

```
[Terminal]
./mvnw -v
./mvnw clean test
./mvnw spring-boot:run
```

3. Recommended package-by-feature MVC structure

```
[Folder tree]
src/main/java/lk/ac/kln/unimart/
  UniMartApplication.java
  common/
    api/
    exception/
    validation/
  config/
    CorsConfig.java
    SecurityConfig.java
  security/
    JwtService.java
    CurrentUser.java
  auth/
    controller/
    dto/
    entity/
    repository/
    service/
  listing/
    controller/
    dto/
    entity/
    repository/
    service/
    mapper/
  review/
    controller/
    dto/
    entity/
    repository/
    service/
    mapper/
  order/
  notification/

src/main/resources/
  application.yml
  application-local.yml
  application-prod.yml
  db/migration/

src/test/java/lk/ac/kln/unimart/
  listing/
  review/
  integration/
```

Organization choice: This is still MVC: controllers handle HTTP, services implement use cases, repositories access data, and React is the view. Package-by-feature keeps each business module cohesive and reduces merge contention.

4. Responsibility rules

Layer	Allowed	Not allowed
Controller	Parse path/query/body, trigger validation, call service, map HTTP status	Direct repository calls, transactions, complex authorization or entity serialization.
Service	Business rules, ownership checks, transactions, orchestration	HTTP-specific objects such as HttpServletRequest unless strictly necessary.
Repository	Query and persist entities	Business workflow decisions.
Entity	Persistence state, simple invariants, relationships	Returning directly from controllers; storing plain-text passwords.
DTO/Mapper	API contract and mapping	Leaking internal fields such as passwordHash or version unless required.

5. Create a minimal health endpoint

```
[Java]
package lk.ac.kln.unimart.common.api;

import java.time.Instant;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/public")
public class PublicController {

  @GetMapping("/ping")
  public Map<String, Object> ping() {
    return Map.of("service", "unimart-backend",
                  "status", "UP",
                  "time", Instant.now());
  }
}
```

Start the application and open http://localhost:8080/api/v1/public/ping. A 200 response proves the web layer runs before database configuration is introduced.

6. Standard API conventions

- Base path: /api/v1. Use plural resource names such as /listings and /reviews.
- Return DTOs, not JPA entities. Use ISO-8601 timestamps in UTC.
- Use 201 Created for successful creation, 204 No Content for successful deletion, 400 for validation, 401 for missing/invalid authentication, 403 for insufficient permission, 404 for absent resources and 409 for state conflicts.
- Use pagination for collection endpoints from the beginning.
- Use one global exception handler with a predictable Problem Details-style response.
- Keep controller methods thin enough to understand in one screen.

```
[Java]
public record ApiError(
    String code,
    String message,
    String path,
    Instant timestamp,
    Map<String, String> fieldErrors
) {}
```

7. Security architecture to prepare

- Passwords are hashed with BCrypt and never logged.
- Public endpoints: registration, login, token refresh, public listing reads and actuator health as configured.
- Protected endpoints: listing write operations, review write operations and user-specific actions.
- JWT access tokens are short-lived. A refresh mechanism is introduced in the integration guide.
- CORS allows only configured frontend origins; wildcard origins are not used with credentials.
- Production secrets come from environment variables or secret stores, never source control.

8. IntelliJ run configuration

7. Open Run > Edit Configurations and create a Spring Boot configuration for UniMartApplication.
8. Set Active profiles to local after application-local.yml is created in Guide 05.
9. Place local environment variables in the configuration or a secure local env-file plugin; do not commit them.
10. Enable Build and run using IntelliJ or Maven consistently across the class. If annotation processing is not used, no extra setting is required.
11. Use breakpoints in Controller -> Service -> Repository order to observe MVC flow.

9. Common setup errors

Error	Cause	Resolution
Unsupported class file / invalid target release	IntelliJ and Maven use different JDKs	Set Project SDK, Maven runner JDK and JAVA_HOME to Java 21.
Port 8080 already in use	Another process is running	Stop it or set SERVER_PORT=8081 for that run.
Maven dependencies unresolved	Offline/proxy/cache problem	Reload Maven; verify proxy; run ./mvnw -U clean test.
401 on /public/ping	Security rules not yet configured	Permit /api/v1/public/** explicitly in SecurityConfig.
Application fails before DB setup	DataSource auto-configuration needs variables	Complete Guide 04-05 or temporarily use a dedicated test profile.

Completion checklist

☐ Project uses Java 21 and Maven Wrapper.

☐ All required Spring dependencies are visible in pom.xml.

☐ Package-by-feature MVC folders exist.

☐ Public ping endpoint returns HTTP 200.

☐ Backend tests run successfully before database integration.

☐ No passwords or secrets are present in application.yml or Git history.